Scikit-learn Preprocessing – தமிழ் Cheat Sheet

ஒரு concept = ஒரு எளிய விளக்கம் + ஒரு example (Tamil explanation, English technical terms)

விளக்கம் = எளிய definition | **உதாரணம்** = code / sample | 💡 Memory trick | ⚠️ கவனிக்க

1 அடிப்படை அமைப்பு (Core Structure)

`Pipeline`

விளக்கம்: பல steps-ஐ ஒன்றன் பின் ஒன்றாக வரிசையில் இணைக்கும் சங்கிலி. ஒரு step-ன் output, அடுத்த step-ன் input ஆகும்.

```
Pipeline([
  ('imputer', SimpleImputer()),
  ('scaler',  StandardScaler())
])
```

💡 Memory trick: Pipeline = தொழிற்சாலை assembly line.

`ColumnTransformer`

விளக்கம்: எந்த column-க்கு எந்த transformer (அல்லது Pipeline) என்று பிரித்து அனுப்பி, கடைசியில் எல்லா output-ஐயும் ஒன்றாகச் சேர்க்கும்.

```
ColumnTransformer([
  ('num', num_pipe, ['age', 'salary']),
  ('cat', OneHotEncoder(), ['city'])
])
```

💡 Memory trick: ColumnTransformer = traffic police. ஒவ்வொரு column-ம் அதற்கான பாதையில் போகும்.

2 காலி இடங்களை நிரப்புதல் (Missing Values)

`SimpleImputer`

விளக்கம்: Data-வில் விடுபட்ட இடங்களை (NaN) ஒரு கணக்கிடப்பட்ட மதிப்பால் நிரப்பும்.

```
SimpleImputer(strategy='median')
```

`strategy = 'mean' | 'median' | 'most_frequent'`

விளக்கம்:

- **mean** = சராசரி (outliers இல்லாத எண்களுக்கு)
- **median** = நடு மதிப்பு (outliers இருந்தால்)
- **most_frequent** = அதிகம் வந்த மதிப்பு (text / category columns-க்கும் வேலை செய்யும்)

```
[10, 20, 30, NaN, 80000]
mean   -> 20015
median -> 25      # outlier 80000 பாதிக்கவி
```

💡 Memory trick: Outlier இருந்தா median, இல்லாட்டி mean.

3 வளைந்த data-வை நேராக்குதல் (Skewness)

`FunctionTransformer + np.log1p`

விளக்கம்: ஒவ்வொரு மதிப்புக்கும் log(1 + x) எடுத்து, வலப்பக்கம் நீண்ட (right-skewed) data-வைச் சுருக்கும்.

```
FunctionTransformer(np.log1p)
log1p(0) = 0          # 0 இருந்தாலும் error
log1p(99) = 4.6
```

⚠️ x > -1 இருக்க வேண்டும். -1 அல்லது அதற்குக் கீழே NaN / -inf வரும்.

`PowerTransformer(method='yeo-johnson')`

விளக்கம்: Negative, 0, positive எல்லாம் கலந்த வளைந்த data-வை Bell Curve (Gaussian) போல மாற்றும்.

```
PowerTransformer(method='yeo-johnson')
# balance: -3000 ... +80000  -> நேராக்கும்
```

⚠️ `standardize=True` default. அதனால் output ஏற்கனவே mean 0, std 1. பிறகு StandardScaler சேர்த்தால் redundant (தேவையில்லை).

💡 Memory trick: Box-Cox = positive மட்டும். Yeo-Johnson = எல்லா எண்களும்.

4 எண்களை scale செய்தல் (Scalers)

`StandardScaler`

விளக்கம்: Mean-ஐ 0 ஆகவும், Std-ஐ 1 ஆகவும் மாற்றும். Formula: (x − mean) / std

```
StandardScaler()
# பெரும்பாலான values சுமார் -3 முதல் +3
```

⚠️ Outliers mean, std-ஐ இழுக்கும். Outliers அதிகம் என்றால் RobustScaler.

`MinMaxScaler`

விளக்கம்: எல்லா மதிப்புகளையும் 0 முதல் 1-க்குள் மாற்றும். Formula: (x − min) / (max − min)

```
MinMaxScaler()
# day: 1 -> 0.0 ,  31 -> 1.0
```

⚠️ ஒரு outlier min / max-ஐ மாற்றி மற்ற values-ஐ அழுத்திவிடும்.

`RobustScaler`

விளக்கம்: Mean / std-க்குப் பதிலாக

median மற்றும் IQR பயன்படுத்தும். Outliers இருந்தாலும் நிலையாக இருக்கும்.

```
RobustScaler()
```

💡 Memory trick: Standard = சராசரியை நம்புது. Robust = நடுவை (median) நம்புது.

5 Text-ஐ எண்களாக மாற்றுதல் (Categorical Encoding)

`OneHotEncoder`

விளக்கம்: Text category column-ஐ, ஒவ்வொரு category-க்கும் ஒரு தனி 0 / 1 column ஆக மாற்றும்.

```
city: Chennai / Coimbatore
-> city_Chennai  city_Coimbatore
        1              0
        0              1
```

`OneHotEncoder parameters`

- **handle_unknown='ignore'** : Test data-வில் புது category வந்தால் error வராது (எல்லா columns-உம் 0).
- **sparse_output=False** : சாதாரண array / table வடிவில் output.
- **dtype=int** : 1.0 அல்ல, 1 என்ற முழு எண்.
- **drop='first'** : மற்ற columns-ல் இருந்தே கணிக்கக்கூடிய ஒரு column-ஐ நீக்கும்.

```
OneHotEncoder(handle_unknown='ignore',
              sparse_output=False,
              dtype=int)
```

⚠️ `drop='first'` linear models-ல் உதவும் (dummy variable trap). Tree models-ல் தேவையில்லை. drop மற்றும் `handle_unknown='ignore'` சேர்த்துப் பயன்படுத்துவதில் version-ஐப் பொறுத்து வரம்புகள் உண்டு, உங்க version-ல் run பண்ணிச் சரிபாருங்க.

6 Column பெயர்கள் & Output அமைப்புகள் (Names & Settings)

`remainder`

விளக்கம்: ColumnTransformer-ல் நாம் குறிப்பிடாத columns-க்கு என்ன செய்வது என்பதைத் தீர்மானிக்கும்.

`feature_names_out="one-to-one"`

விளக்கம்: **Parameter** (விதி). FunctionTransformer-ல் input-ல் இருந்த அதே எண்ணிக்கை, அதே பெயர்கள்

• **'drop'** (default) = நீக்கிவிடும்
• **'passthrough'** = மாற்றாமல், output-ன் கடைசியில் சேர்க்கும்

```
ColumnTransformer([...], remainder='passthrou
```

💡 Memory trick: Drop = குப்பைத் தொட்டி. Passthrough = பின்வரிசை டிக்கெட்.

output-லும் வரும் என்று sklearn-க்கு உறுதி கொடுக்கும்.

```
FunctionTransformer(np.log1p,
    feature_names_out="one-to-one")
# 'duration' -> 'duration'
```

⚠️ இது இல்லையென்றால் `get_feature_names_out()` இந்த transformer-க்கு available இல்லை. Function columns-ன் எண்ணிக்கையை மாற்றினால் "one-to-one" தவறு.

`get_feature_names_out()`

விளக்கம்: Method (விசாரணை). Fit ஆன பிறகு, output columns-ன் பெயர்களைத் திருப்பித் தரும்.

```
ct.fit(df)
ct.get_feature_names_out()
# ['num__age', 'num__salary', 'cat__city_Cher
```

💡 Memory trick: Parameter = சட்டம் எழுதுவது. Method = சட்டத்தைப் படிப்பது.

`set_output / set_config (pandas)`

விளக்கம்: Transformer output-ஐ NumPy array-க்குப் பதிலாக DataFrame ஆக்கும். Column names, `get_feature_names_out()`-ல் இருந்து எடுக்கப்படும்.

```
ct.set_output(transform="pandas")        # ஒ
set_config(transform_output="pandas")    # gl
```

⚠️ Names prefix-உடன் (`num__age`) வரலாம். Original name அப்படியே இருக்கும் என்று உறுதி இல்லை.

`verbose_feature_names_out=False`

விளக்கம்: ColumnTransformer-ன் parameter. Output names-ல் transformer name prefix சேர்க்காமல் சுத்தமாக வைக்கும்.

```
True  (default) -> 'num__balance'
False           -> 'balance'
```

⚠️ இரண்டு transformers ஒரே பெயரை உருவாக்கினால் error வரும்.

`fit_transform(X_train) vs transform(X_test)`

விளக்கம்:
• **fit_transform** = Train data-வில் இருந்து கற்றுக்கொண்டு (mean, median, categories) மாற்றும்.
• **transform** = கற்றுக்கொண்டதை வைத்து Test data-வை மட்டும் மாற்றும்.

```
X_train_t = ct.fit_transform(X_train)
X_test_t  = ct.transform(X_test)
```

⚠️ Test data-வில் fit பண்ணக்கூடாது. அது

data leakage.

🗺️ Pipeline-க்குள் names எப்படிப் போகும்? (Relay Race)

```
ct.get_feature_names_out()
 │
 ├─ "num" (Pipeline)   input names: ['age', 'salary']
 │     imputer    ['age','salary'] ──► ['age','salary']
 │     transform  ['age','salary'] ──► ['age','salary']
 │     scalar     ['age','salary'] ──► ['age','salary']   ← கடைசி step-ன் names மட்டும் வெளியே
 │
 └─ "cat" (OneHotEncoder)  ['city'] ──► ['city_Chennai', 'city_Coimbatore']

இறுதி: ['num__age', 'num__salary', 'cat__city_Chennai', 'cat__city_Coimbatore']
```

💡 Memory trick: Baton = names list. ColumnTransformer, Pipeline-இடம் மட்டும் கேட்கும். Pipeline உள்ளே baton-ஐக் கைமாற்றும். Step names (`imputer`, `scalar`) final name-ல் வராது.

❓ Interview Quick Q&A

Q1. `remainder` mention பண்ணாவிட்டால் என்ன ஆகும்?
A: Default 'drop'. குறிப்பிடாத columns நீங்கி விடும், error வராது.

Q2. `feature_names_out` parameter-க்கும் `get_feature_names_out()` method-க்கும் வித்தியாசம்?
A: Parameter = names எப்படி உருவாக வேண்டும் என்ற விதி (constructor-ல்). Method = fit-க்குப் பிறகு அந்த names-ஐக் கேட்டு வாங்குவது.

Q3. Outliers அதிகம் உள்ள column-க்கு எந்த scaler?
A: RobustScaler (median, IQR). StandardScaler-ல் outliers mean, std-ஐ இழுக்கும்.

Q4. Test data-வில் ஏன் `fit_transform` பயன்படுத்தக்கூடாது?
A: Test data-வில் இருந்து கற்றுக்கொண்டால் data leakage ஆகும். Train-ல் கற்றதை வைத்து `transform` மட்டும்.

Q5. Pipeline-ல் ஒரு `FunctionTransformer`-ல் `feature_names_out` இல்லையென்றால்?
A: Names கேட்கும் இடத்தில் error வரும். "one-to-one" அல்லது callable கொடுத்துச் சரிசெய்யலாம்.

Remember: Parameter = write the rule · Method = read the names